

GROUP1 ADVANCED

Cloud & DevSecOps Capstone Project

Secure software delivery from code to
validated development deployment

PYTHON

TERRAFORM

DOCKER

DEVSECO
PS

Code → Test → Secure → Build → Release → Deploy →
Validate



Project overview

A practical capstone focused on repeatable delivery, infrastructure validation, and built-in security.

Application

Python API
Health and documentation endpoints
Automated pytest validation

Infrastructure

Terraform definitions
Formatting and validation gates
Network, logging and encryption controls

Delivery

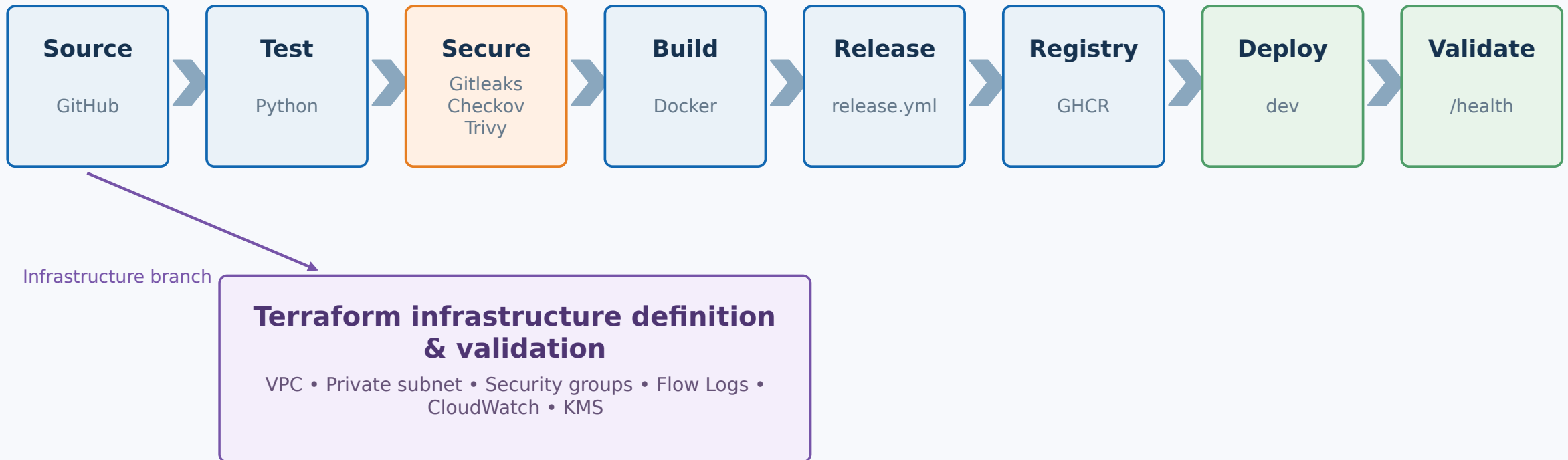
GitHub Actions CI/CD
Container build and registry release
Dev deployment and health checks

Outcome

A working, security-focused delivery pipeline with evidence at every stage.

Solution architecture

The primary delivery flow is linear, with infrastructure validation performed alongside application CI.



Code → Test → Secure → Build → Release → Deploy → Validate

Python API & automated tests

The application exposes simple endpoints that are exercised locally and inside the deployment workflow.

API surface

GET / - root response
GET /health - service health
GET /doc - documentation

Test coverage

Pytest health endpoint
Pytest documentation endpoint
Syntax and import validation

Observed evidence

Local pytest: 2 passed
Browser health check: status = ok
CI application tests: passed

Health validation is a deployment gate, not only a local test.

Infrastructure as Code

Terraform is treated as a controlled, testable infrastructure definition in the delivery process.

Defined controls

AWS VPC
Private subnet
Security groups
VPC Flow Logs

Logging & encryption

CloudWatch logging
365-day retention
KMS encryption
Key rotation

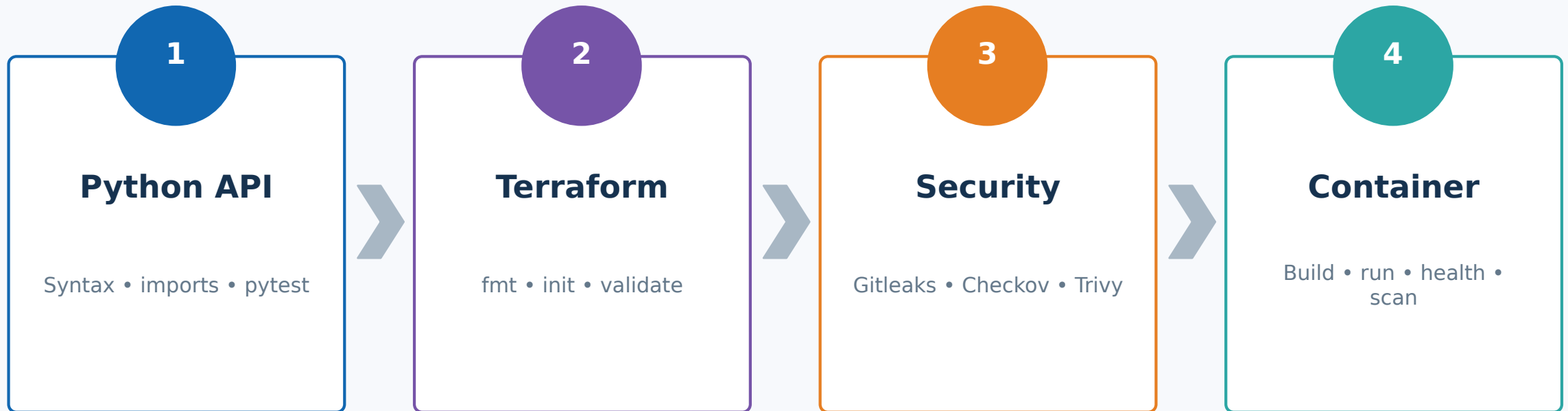
Pipeline gates

```
terraform fmt -check  
terraform init -backend=false  
terraform validate  
Checkov IaC scan
```

Scope note: this capstone demonstrates infrastructure definition and validation; the diagram does not claim live AWS provisioning.

Continuous integration pipeline

ci.yml blocks progression unless application, infrastructure, security, and container checks succeed.



Result: all CI jobs passed

DevSecOps security controls

Security is integrated into the pipeline as a set of explicit quality gates.

Gitleaks

- Detects committed secrets
- Runs against repository history
- Blocks release on findings

Checkov

- Scans Terraform IaC
- Identifies configuration risks
- Supports documented controls

Trivy

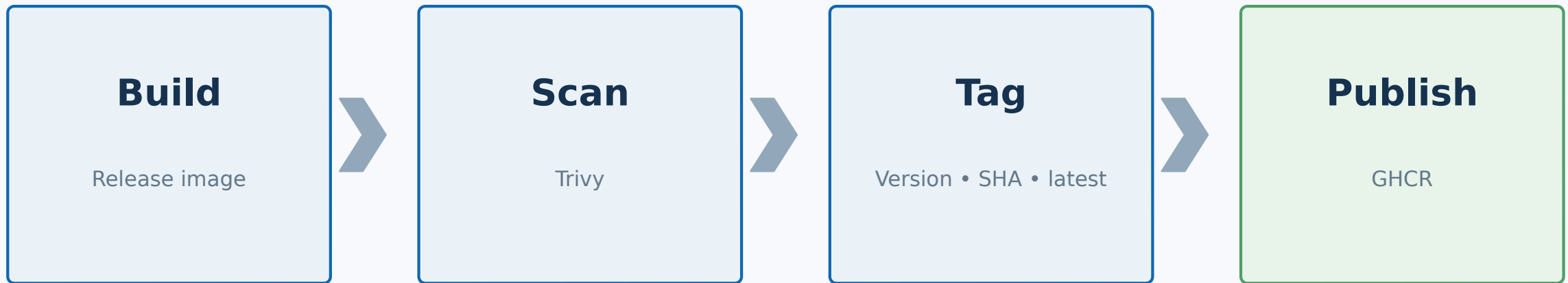
- Filesystem vulnerability scan
- Container image scan
- HIGH / CRITICAL gate

Security
principle

Detect issues before an image reaches the deployment stage.

Release & container registry

release.yml creates a deployable artifact only after a successful security scan.



Registry artifact becomes the controlled input to deployment.

Development deployment

deploy.yml uses the dev environment, pulls the released image, starts the container, and verifies the API.

Deploy

GitHub environment: dev
Authenticate to GHCR
Pull selected tag
Run container on port 8080

Validate

Wait for application
GET /health
GET /
GET /doc

Observed result

Deployment workflow passed
Health endpoint returned status: ok
Cleanup runs after validation

DEV DEPLOYMENT SUCCESSFUL

Testing & evidence

Submission evidence demonstrates both local behavior and automated pipeline results.



Local tests

2 passed



Health API

status: ok



CI

Passed



Release

Passed



Deploy dev

Passed

Evidence set: repository structure • pytest • /health • CI • security • release • dev deployment • architecture

Key challenges & resolutions

The implementation demonstrates iterative troubleshooting across infrastructure, containers, security tooling, and registry delivery.

Terraform formatting



Applied fmt checks and validation before progression

Container image visibility



Loaded the Buildx image for local Trivy scanning

GHCR image naming



Normalized the container repository path to lowercase

Image tag resolution



Aligned release and deployment image references

Deployment confidence



Added health and API verification after container startup

Conclusion

The capstone demonstrates a complete, evidence-driven DevSecOps delivery workflow.

Automated

Tests and repeatable workflows

Secure

Secret, IaC and vulnerability scanning

Portable

Containerized release artifact

Controlled

Versioned registry publishing

Verified

Post-deployment API health checks

READY FOR CAPSTONE SUBMISSION

Code → Test → Secure → Build → Release → Deploy → Validate